© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_9

9. Prompt Alchemy: Patterns That Make Models Look Smarter

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

The goal of this chapter is to explore the art of prompt engineering using Spring AI.

Introduction

This chapter into the fascinating world of prompt engineering using Spring AI. As artificial intelligence continues to evolve, the ability to effectively communicate with large language models (LLMs) has become a crucial skill. A *prompt*, the input text given to an AI model, plays a significant role in shaping the output. By carefully crafting these prompts, you can guide the AI to generate more accurate, relevant, and context-aware responses.

Let's start by exploring the concept of a prompt and the importance of prompt engineering. From there, we will dive into the patterns and techniques that can be applied to create effective prompts, including simple approaches like text summarization and question answering, as well as advanced methods like zero-shot, few-shot learning, and ReAct. The chapter will also introduce how to use OpenAI's models through the Ollama API in Spring AI, making it possible to harness the power of these sophisticated models without the need for complex setup or additional licenses.

By the end of this chapter, you'll have a solid understanding of prompt engineering and how to leverage Spring AI to integrate LLMs into your applications, optimizing interactions with AI for a wide range of use cases.

What Is a Prompt?

In the context of LLMs, a prompt is the input text that you provide to the model in order to generate the desired output. The prompt acts as a request or instruction for the model, helping it predict what the next sequence of words or tokens should be. In essence, a prompt sets the stage for the model's response based on the patterns it has learned during its training on large datasets.

Creating effective prompts can be a nuanced task, as various factors—such as the wording, structure, context, and specific instructions—can significantly influence the quality and relevance of the output. The process of designing these prompts to elicit accurate, useful, and appropriate responses is known as *prompt engineering*.

What Is Prompt Engineering?

Prompt engineering is the process of designing and optimizing input prompts to effectively guide LLMs like GPT or Gemini to generate the desired outputs. Since LLMs predict text based on patterns in the input they receive, the way you phrase your prompt significantly impacts the quality and accuracy of their responses.

This process involves several key activities, such as:

1. **Choosing the right prompt structure**: You can use different prompting techniques like zero-shot (where the model is given a task without any examples), one-shot (with one example), or few-shot (with multiple examples). These structures help the model better understand the task at hand.
2. **Configuring the model's settings**: Various settings, like temperature (which controls randomness), top-K (which restricts the model to select from the top-K most probable tokens), and top-P (which ensures the selection of tokens whose cumulative probability doesn't exceed a threshold), influence how the model generates its responses.
3. **Tailoring the prompt for context**: The prompt can be adjusted to provide specific context, instructions, or even role assignments (e.g., asking the model to act as a teacher, travel guide, or code reviewer) to improve the quality of the output.
4. **Iteration and evaluation**: Prompt engineering is an iterative process. You may need to experiment with different word choices, styles, and formats to refine the prompt for optimal results. You also evaluate the responses to ensure they meet your expectations, tweaking the prompts as needed.

Ultimately, prompt engineering helps you fine-tune the interaction with the language model to ensure it understands the task correctly and generates responses that align with your needs.

Prompt Engineering Patterns

Prompt engineering is a critical process for crafting input queries that guide LLMs in producing the desired output. Effective prompts can be complex and require a clear understanding of both the AI's capabilities and the task at hand. Based on the content of the attached document, the following are the prompt engineering practices and patterns that can help achieve this.

Creating Effective Prompts

To create effective prompts, it is essential to integrate several key components to ensure clarity and maximize the likelihood of receiving accurate, relevant responses.

Key Components of a Prompt

These are the key components of a prompt:

- **Instructions**: Giving clear, concise instructions is the foundation of an effective prompt. This is akin to explaining the task directly to the AI, as you would to a person. A well-defined instruction helps guide the AI's response in the right direction. For example, instead of asking a vague question like "Tell me about this," a more effective prompt would be "Summarize the key points of this text."
- **External context**: Providing additional context or background information helps the AI understand the scenario better and provides it with the necessary framework to generate a more informed response. For example, in code generation tasks, including the specific programming language and purpose of the code can ensure the response is tailored to those needs.
- **User input**: The core of the prompt is the user's request or question. The AI should understand exactly what the user is asking for. For example, in a question-answering task, the user input could be something like "What is the capital of France?"
- **Output indicator**: Specifying the desired format for the output is crucial, but the AI might not always comply strictly. For example, if you request a JSON response, the AI might prepend a phrase like "Here is your JSON" before the actual data. Despite this, clarifying the format you need (like JSON, plain text, or a list) helps guide the AI.

Examples of Expected Output Format

These are some example:

- Providing examples of how you expect the question and answer format to look (such as structured data or a formal tone) can further enhance the precision of the output. This sets a clear expectation for the AI, making it easier for the model to match your query's intent.

Simple Prompting Techniques

There are several foundational prompting techniques commonly used in AI applications, including:

- **Text summarization**: This reduces extensive text into a concise version, highlighting key points and omitting less critical details. This is particularly useful in cases where the user has large volumes of text they want to distill into manageable summaries.
- **Question answering**: The AI responds to a question by extracting specific information from a provided text. Effective question answering requires the AI to understand the context and identify relevant information that directly addresses the user's query.
- **Text classification**: The model categorizes text into predefined groups. For example, it could classify a review as "positive," "negative," or "neutral" based on the sentiment expressed in the text.
- **Conversation**: This involves simulating a natural back-and-forth dialogue. The AI maintains conversational flow, which is ideal for customer service bots or interactive applications where user engagement is critical.
- **Code generation**: The model generates executable code snippets based on a natural language description. This allows users to describe coding needs in simple terms and receive code tailored to their specifications.

Advanced Prompting Techniques

Advanced techniques enhance the model's ability to understand more complex tasks, often requiring more detailed reasoning or examples:

- **Zero-shot and few-shot learning**: Zero-shot learning refers to making accurate predictions with little to no prior examples, while few-shot learning uses a small set of examples to help the model generalize to new tasks. These methods allow the AI to handle tasks with minimal direct guidance.

- **Chain-of-thought (CoT)**: This technique involves the model reasoning step-by-step through a problem, which helps maintain the context and ensures that the output is coherent and logically structured. It's especially useful in complex problem-solving or tasks that require a logical progression, such as math or logic problems.
- **ReAct (reason + act)**: In ReAct prompting, the model first reasons about the task at hand and then determines the most appropriate action to take. This approach combines analytical thinking with decision-making and is particularly useful in tasks that require both reasoning and external actions (like querying a database or retrieving information).

Microsoft's Framework for Prompt Creation and Optimization

Microsoft provides a structured framework for creating and refining prompts. This framework emphasizes the following:

- **Clarity**: Ensuring that the prompt is clear and unambiguous
- **Specificity**: Being as specific as possible in your instructions to guide the model's response
- **Iterative Refinement**: Continuously improving the prompt by evaluating the AI's responses and tweaking the input accordingly
- **Contextualization**: Providing the AI with as much relevant context as possible to make its response more accurate and insightful

By following this framework, users can better optimize the prompts they create, ensuring they align closely with the intended outcome.

Best Practices for Prompt Engineering

The document suggests several best practices to follow for effective prompt engineering:

- **Provide examples**: Use one-shot or few-shot prompting to provide examples of the task, guiding the AI to understand the structure and tone required for the response.
- **Design with simplicity**: Keep prompts clear and direct. Avoid overcomplicating instructions that might confuse both the AI and the user.
- **Be specific about the output**: Always specify the type and structure of the response you expect, whether it's a specific format like JSON or a particular style.
- **Use instructions over constraints**: Focus on positive instructions that clarify what the AI should do instead of listing things it should avoid. This promotes creativity within boundaries, rather than restricting the AI's potential.
- **Control the max token length**: For tasks requiring concise output, set a token length limit to avoid overly verbose or irrelevant responses.

These techniques and best practices allow for the development of more effective prompts that yield the most accurate and relevant responses from AI models.

Using Prompt Engineering in Spring AI

With the basic setup from Chapter **4** using Ollama with OpenAI as the foundation, you can explore the world of prompt engineering through practice.

Reviewing the flow in Figure 9-1, you can see four layers: Prompt, ChatModel, Native LLM API, and ChatResponse. Prompt, ChatModel, and ChatResponse are layers specific to Spring AI, while Native LLM API is specific to the LLM API interface. The flow starts with setting the instructions and the chat options. These are specific to the Prompt layer, and we will see in detail later how these are represented in the code. The Prompt is set on the ChatModel, which internally converts the input to the format applicable to specific LLM APIs. This is one of the primary advantages of using Spring AI, which creates these helpful abstractions. The LLM then responds with a response, and the response is converted to the ChatResponse Spring AI format.

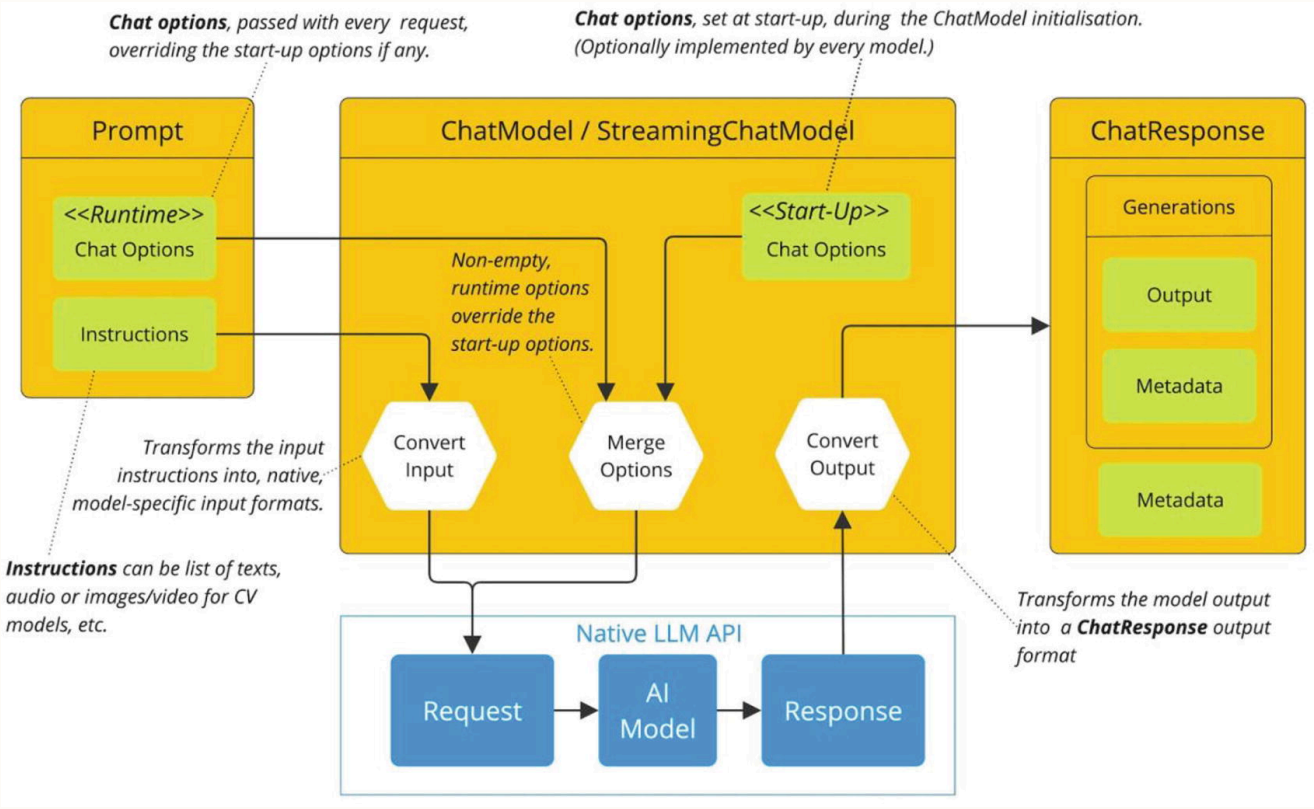


Figure 9-1 Inner workings of prompt engineering

Before exploring prompt engineering methods, it's important to first learn how to adjust the LLM's output settings. Spring AI offers a range of configuration options via the ChatOptions builder, allowing you to fine-tune various generation parameters. You can apply these settings directly in your code—as shown in the following examples—or define them in your Spring application properties at startup.

Let's do some foundational setup before we proceed on to each example. We will create a CommandLineRunner bean, which will get executed whenever our application boots up. The ChatClient API is foundational to all our examples, so we will create an instance of it using the OpenAiChatModel, which is injected at runtime by Spring AI. This allows us to interact with the OpenAI API. Every other example we build upon will be created in a new private method that will be taken as input to the chatClient and called from within the CommandLineRunner bean.

File: src/main/java/com/example/demo/DemoApplication.java

```
@Bean
public CommandLineRunner commandLineRunner(OpenAiChatModel openAiChatModel) {
   return args -> {
      ChatClient chatClient = ChatClient.builder(openAiChatModel).build();
   }
}
```

Temperature

Temperature controls the randomness or "creativity" of the model's response.

- **Lower values (0.0-0.3)**: More deterministic, focused responses. Better for factual questions, classification, or tasks where consistency is critical.

```
@Bean
public CommandLineRunner commandLineRunner(OpenAiChatModel openAiChatModel) {
   return args -> {
      ChatClient chatClient = ChatClient.builder(openAiChatModel).build();
      lowerValueTemperaturePrompting(chatClient);
   }
}
private void lowerValueTemperaturePrompting(ChatClient chatClient) {
   ChatOptions chatOptions = ChatOptions.builder()
         .model("gemma3:1b")
         .maxTokens(100)
         .temperature(0.1)
         .build();
   String result = chatClient.prompt("A short poem on life of Ducks in space.")
         .options(chatOptions)
         .call()
         .content();
   System.out.println("Result: " + result);
}
```

Output:

```
Result: Okay, here's a short poem about the life of ducks in space, aiming for a slightly whimsical and contemplative feel:
---
In orbit's grace, a feathered flight,
Ducks drift in darkness, bathed in light.
No pond to splash, no willow tree,
Just stardust trails for all to see.
They ponder planets, blue and grand,
A cosmic ballet, close at hand.
Collecting moonbeams, soft and slow,
Duck life in
```

- **Medium values (0.4-0.7)**: Balanced between determinism and creativity. Good for general use cases.
  Let's change the temperature parameter and see how the output changes.

```
ChatOptions chatOptions = ChatOptions.builder()
      .model("gemma3:1b")
      .maxTokens(100)
      .temperature(0.5)
      .build();
```

Output:

```
Result: Okay, here's a short poem about the life of ducks in space, aiming for a slightly whimsical and contemplative feel:
---
In orbit's grace, a feathered flight,
Ducks drift in darkness, bathed in light.
No pond to swim, no reeds to roam,
Just stardust trails, a cosmic home.
They bounce and glide, a silent grace,
Observing Earth, a distant space.
A tiny wonder, sleek and bold,
A
```

- **Higher values (0.8-1.0)**: More creative, varied, and potentially surprising responses. Better for creative writing, brainstorming, or generating diverse options.
  Let's now try to increase the temperature more to 1.0 value.

```
ChatOptions chatOptions = ChatOptions.builder()
      .model("gemma3:1b")
      .maxTokens(100)
```

```
        .temperature(1.0)
        .build();
```

Output:

```
Result: Okay, here's a short poem about the life of ducks in space, aiming for a slightly whimsical and evocative feel:
**Orbit's Soft Landing**
Small wings, a moonlit gleam,
Ducks drift in a cosmic dream.
No pond to dip, no earth below,
Just stardust trails and silent snow.
They float and ponder, weightless grace,
A tiny kingdom in space's embrace.
Observing nebulae, bright and bold
```

You can observe the change in the text, with it being a bit more creative. Understanding temperature is crucial for prompt engineering as different techniques benefit from different temperature settings.

Output Length (MaxTokens)

The maxTokens parameter limits how many tokens (word pieces) the model can generate in its response.

- **Low values (5-25)**: For single words, short phrases, or classification labels
- **Medium values (50-500)**: For paragraphs or short explanations
- **High values (1000+)**: For long-form content, stories, or complex explanations

Previously we limited the maxTokens to 100 and changed the prompt to "A poem" instead of "A short poem." Let's increase it and see the response. You might see a more complete and longer poem.

```
ChatOptions chatOptions = ChatOptions.builder()
        .model("gemma3:1b")
        .maxTokens(500)
        .temperature(1.0)
        .build();
String result = chatClient.prompt("A poem on life of ducks in space.")
        .options(chatOptions)
        .call()
        .content();
System.out.println("Result: " + result);
```

Setting the appropriate output length is important to ensure you get a complete responses without unnecessary verbosity.

These parameters give you fine-grained control over the token selection process during generation:

- **Top-K**: Limits token selection to the K most likely next tokens. Higher values (e.g., 40-50) introduce more diversity.
- **Top-P (nucleus sampling)**: Dynamically selects from the smallest set of tokens whose cumulative probability exceeds P. Values like 0.8-0.95 are common.

```
.options(ChatOptions.builder()
        .topK(40)     // Consider only the top 40 tokens
        .topP(0.8)    // Sample from tokens that cover 80% of probability mass
        .build())
```

These sampling controls work in conjunction with temperature to shape response characteristics. Please note that not all of the options are applicable to OpenAI since some of the LLM APIs may not have these as options. Therefore, always refer to the documentation or the debug messages when you run your application.

Structured Response Format

Along with the plain-text response (using .content()), Spring AI makes it easy to directly map LLM responses to Java objects using the .entity() method.

Say you have defined a record object named ActorFilms with two properties called actor and movies. Now you will create a prompt to ask OpenAI to generate the filmography for a random actor. The output from OpenAI can be encoded to the ActorFilms record entity type so that it can be used directly in the application.

```
record ActorFilms(String actor, List<String> movies) {}
@SpringBootApplication
public class DemoApplication {
  public static void main(String[] args) {
      SpringApplication.run(DemoApplication.class, args);
  }
  private void outputFormat(ChatClient chatClient) {
      ActorFilms actorFilms = chatClient
            .prompt("Generate the filmography for a random actor.")
            .call()
            .entity(ActorFilms.class);
     System.out.println("Output: " + actorFilms);
  }
  @Bean
   public CommandLineRunner commandLineRunner(OpenAiChatModel openAiChatModel) {
      return args -> {
         ChatClient chatClient = ChatClient.builder(openAiChatModel).build();
         outputFormat(chatClient);
      }
  }
}
Started DemoApplication in 4.896 seconds (process running for 5.43)
Output: ActorFilms[actor=Jamie Lee Curtis, movies=[Trick or Treat, Cutthroat Earth, Wes Craven's New Nightmare, Dark Shadows, Step Brothers, The Sixth Sense
```

This feature is particularly powerful when combined with system prompts that instruct the model to return structured data.

Model-Specific Options

While the portable ChatOptions provides a consistent interface across different LLM providers, Spring AI also offers model-specific options classes that expose provider-specific features and configurations. These model-specific options allow you to leverage the unique capabilities of each LLM provider.

```
// Using OpenAI-specific options
private void openAiSpecificOptions(ChatClient chatClient) {
   OpenAiChatOptions openAiOptions = OpenAiChatOptions.builder()
         .model("gemma3:1b")
         .temperature(0.2)
         .frequencyPenalty(0.5)   // OpenAI-specific parameter
         .presencePenalty(0.3)    // OpenAI-specific parameter
         .seed(42)              // OpenAI-specific deterministic generation
         .build();
  String result = chatClient.prompt("Share the stock value of Apple Inc.")
         .options(openAiOptions)
         .call()
         .content();
   System.out.println("Result: " + result);
}
@Bean
public CommandLineRunner commandLineRunner(OpenAiChatModel openAiChatModel) {
   return args -> {
      ChatClient chatClient = ChatClient.builder(openAiChatModel).build();
      openAiSpecificOptions(chatClient);
   }
}
```

Output:

```
Result: As of today, November 2, 2023, at 8:30 AM PST, Apple Inc. (AAPL) stock is trading at approximately **$177.87 per share**.
**You can find the most up-to-date and precise stock value on these resources:**
*   **Google Finance:** [https://www.google.com/finance/quote/AAPL:NASDAQ](https://www.google.com/finance/quote/AAPL:NASDAQ)
*   **Yahoo Finance:** [https://finance.yahoo.com/quote/AAPL/](https://finance.yahoo.com/quote/AAPL/)
*   **Bloomberg:** [https://www.bloomberg.com/quote/AAPL](https://www.bloomberg.com/quote/AAPL)
```

Disclaimer: *I am an AI Chatbot and not a financial advisor. This information is for general knowledge and informational purposes only, and does not constitut

Each model provider has its own implementation of chat options (e.g., OpenAiChatOptions, AnthropicChatOptions, MistralAiChatOptions) that expose provider-specific parameters while still implementing the common interface. This approach gives you the flexibility to use portable options for cross-provider compatibility or model-specific options when you need access to unique features of a particular provider.

Note that when using model-specific options, your code becomes tied to that specific provider, reducing portability. It's a trade-off between accessing advanced provider-specific features versus maintaining provider independence in your application.

Prompt Engineering Techniques

Each of the following sections implements a specific prompt engineering technique from the guide. By following both the "Prompt Engineering" guide and these implementations, you'll develop a thorough understanding of not just what prompt engineering techniques are available but how to effectively implement them in production Java applications.

Zero-Shot Prompting

Zero-shot prompting involves asking an AI to perform a task without providing any examples. This approach tests the model's ability to understand and execute instructions from scratch. Large language models are trained on a vast corpora of text, allowing them to understand what tasks like “translation,” “summarization,” or “classification” entail without explicit demonstrations.

Zero-shot is ideal for straightforward tasks where the model likely has seen similar examples during training and when you want to minimize prompt length. However, performance may vary depending on task complexity and how well the instructions are formulated.

```
private void zeroShotPrompting(ChatClient chatClient) {
    String reviewSentiment = chatClient.prompt("""
    Classify movie reviews as POSITIVE, NEUTRAL or NEGATIVE.
    Review: "Her" is a disturbing study revealing the direction
    humanity is headed if AI is allowed to keep evolving,
    unchecked. I wish there were more movies like this masterpiece.
    Sentiment:
    """)
        .options(ChatOptions.builder()
            .model("gemma3:1b")
            .temperature(0.1)
            .maxTokens(5)
            .build())
        .call()
        .content();
    System.out.println("Output: " + reviewSentiment);
}
```

Output:

```
Started DemoApplication in 5.285 seconds (process running for 5.841)
Output: Negative
**Reasoning
```

This example shows how to classify a movie review sentiment without providing examples. Note the low temperature (0.1) for more deterministic results.

One-Shot and Few-Shot Prompting

Few-shot prompting provides the model with one or more examples to help guide its responses, particularly useful for tasks requiring specific output formats. By showing the model examples of desired input-output pairs, it can learn the pattern and apply it to new inputs without explicit parameter updates.

One-shot provides a single example, which is useful when examples are costly or when the pattern is relatively simple. Few-shot uses multiple examples (typically three to five) to help the model better understand patterns in more complex tasks or to illustrate different variations of correct outputs.

```
private void fewShotPrompting(ChatClient chatClient) {
```

```
    String pizzaOrder = chatClient.prompt("""
      Parse a customer's pizza order into valid JSON
      EXAMPLE 1:
      I want a small pizza with cheese, tomato sauce, and pepperoni.
      JSON Response:
      ```
      {
         "size": "small",
         "type": "normal",
         "ingredients": ["cheese", "tomato sauce", "pepperoni"]
      }
      ```
      EXAMPLE 2:
      Can I get a large pizza with tomato sauce, basil and mozzarella.
      JSON Response:
      ```
      {
         "size": "large",
         "type": "normal",
         "ingredients": ["tomato sauce", "basil", "mozzarella"]
      }
      ```
      Now, I would like a large pizza, with the first half cheese and mozzarella.
      And the other tomato sauce, ham and pineapple.
      """)
          .options(ChatOptions.builder()
                .model("gemma3:1b")
                .temperature(0.1)
                .maxTokens(250)
                .build())
          .call()
          .content();
    System.out.println("Output: " + pizzaOrder);
}
```

Output:

```
Started DemoApplication in 4.541 seconds (process running for 5.029)
Output:
```json
{
  "size": "large",
  "type": "normal",
  "ingredients": ["cheese", "mozzarella"]
}
```
```

Few-shot prompting is especially effective for tasks requiring specific formatting, handling edge cases, or when the task definition might be ambiguous without examples. The quality and diversity of the examples significantly impact performance.

System, Contextual, and Role Prompting

In this section, we will discuss about prompting in the context of system, contextual and role scopes and how they are applicable.

System Prompting

System prompting sets the overall context and purpose for the language model, defining the "big picture" of what the model should be doing. It establishes the behavioral framework, constraints, and high-level objectives for the model's responses, separate from the specific user queries.

System prompts act as a persistent "mission statement" throughout the conversation, allowing you to set global parameters like output format, tone, ethical boundaries, or role definitions. Unlike user prompts that focus on specific tasks, system prompts frame how all user prompts should be interpreted.

```
private void systemPrompting(ChatClient chatClient) {
    String movieReview = chatClient
```

```
        .prompt()
        .system("Classify movie reviews as positive, neutral or negative. Only return the label in uppercase.")
        .user("""
          Review: "Her" is a disturbing study revealing the direction
          humanity is headed if AI is allowed to keep evolving,
          unchecked. It's so disturbing I couldn't watch it.
          Sentiment:
          """)
        .options(ChatOptions.builder()
            .model("gemma3:1b")
            .temperature(1.0)
            .topP(0.8)
            .maxTokens(5)
            .build())
        .call()
        .content();
    System.out.println("Output: " + movieReview);
}
```

Output:

```
Started DemoApplication in 4.834 seconds (process running for 5.341)
Output: NEGATIVE
```

System prompting is particularly powerful when combined with Spring AI's entity mapping capabilities:

```
private void systemPrompting2(ChatClient chatClient) {
    record MovieReviews(Movie[] movie_reviews) {
        enum Sentiment {
            POSITIVE, NEUTRAL, NEGATIVE
        }
        record Movie(Sentiment sentiment, String name) {
        }
    }
    MovieReviews movieReviews = chatClient
        .prompt()
        .system("""
        Classify movie reviews as positive, neutral or negative. Return
        valid JSON.
        """)
        .user("""
        Review: "Her" is a disturbing study revealing the direction
        humanity is headed if AI is allowed to keep evolving,
        unchecked. It's so disturbing I couldn't watch it.
        JSON Response:
        """)
        .call()
        .entity(MovieReviews.class);
    System.out.println("Output: " + Arrays.stream(movieReviews.movie_reviews()).findFirst());
}
```

Output:

```
Started DemoApplication in 4.706 seconds (process running for 5.206)
Output: Optional[Movie[sentiment=NEGATIVE, name=null]]
```

System prompts are especially valuable for multiturn conversations, ensuring consistent behavior across multiple queries, and for establishing format constraints like JSON output that should apply to all responses.

Role Prompting

Role prompting instructs the model to adopt a specific role or persona, which affects how it generates content. By assigning a particular identity, expertise, or perspective to the model, you can influence the style, tone, depth, and framing of its responses.

Role prompting leverages the model's ability to simulate different expertise domains and communication styles. Common roles include expert (e.g., "You are an experienced data scientist"), professional (e.g., "Act as a travel guide"), or stylistic character (e.g., "Explain like you're Shakespeare").

```
private void rolePrompting(ChatClient chatClient) {
    String travelSuggestions = chatClient
            .prompt()
            .system("""
              I want you to act as a travel guide. I will write to you
              about my location and you will suggest 3 places to visit near
              me. In some cases, I will also give you the type of places I
              will visit.
              """)
            .user("""
              My suggestion: "I am in Amsterdam and I want to visit only museums."
              Travel Suggestions:
              """)
            .call()
            .content();
    System.out.println("Output: " + travelSuggestions);
}
```

Output:

```
Started DemoApplication in 3.902 seconds (process running for 4.372)
Output: Okay, fantastic! Amsterdam is a museum lover's paradise. Here are 3 suggestions, keeping in mind you're primarily focusing on museums:
1. **The Rijksmuseum:** (Art & History) - This is *the* iconic museum in Amsterdam. It houses a breathtaking collection of Dutch Masters - Rembrandt, Vermeer,
2. **Van Gogh Museum:** (Art) - If you're a fan of Vincent van Gogh, this museum is a pilgrimage. It's home to the largest collection of his works and offers a dee
3. **Stedelijk Museum:** (Modern & Contemporary Art) - This museum focuses on 20th and 21st-century art, design, and architecture. It has a fantastic collection
To help me refine these suggestions even further, could you tell me:
*   **What kind of art/historical periods are you most interested in?** (e.g., Dutch Masters, Impressionism, Modern Art, etc.)
```

Role prompting can be enhanced with style instructions:

```
private void rolePrompting2(ChatClient chatClient) {
    String humorousTravelSuggestions = chatClient
            .prompt()
            .system("""
              I want you to act as a travel guide. I will write to you about
              my location and you will suggest 3 places to visit near me in
              a humorous style.
              """)
            .user("""
              My suggestion: "I am in Amsterdam and I want to visit only museums."
              Travel Suggestions:
              """)
            .call()
            .content();
    System.out.println("Output: " + humorousTravelSuggestions);
}
```

Output:

```
Started DemoApplication in 3.983 seconds (process running for 4.479)
Output: Alright, Amsterdam! Excellent choice - you've got a serious museum obsession! Let's get you prepped for a delightfully cultured trip. Here are three place
**1. The Rijksmuseum - Prepare for a Deep Dive into Royal Portraits (and Slightly Boring Paintings):**
Seriously, you *need* to see this. Think powdered wigs, velvet gowns, and a whole lot of people trying to look regal. It's basically a walking, talking history lesson
**Why it's hilarious:** It's the embodiment of Dutch history, but it's also a giant, beautifully decorated room filled with portraits of boring dudes.
**2. The Van Gogh Museum -  Embrace the Madness (and the Slightly Chaotic):**
Okay, this one's a little... intense.  Prepare for a whirlwind of swirling colors, melancholic faces, and the sheer, unadulterated *energy* of Van Gogh. It's like a fev
**Why it's hilarious:** It's a monument to a troubled genius.  You'll think you're witnessing the birth of a masterpiece, but it's really just a guy painting a lot of thi
**3. The ABBA The Museum -  For the Nostalgic (and Slightly Dramatic) Traveler:**
Yes, *yes*, I know. ABBA. But hear me out. This museum is a *massive* celebration of the Swedish pop sensation. Seriously, it's a shrine to sparkly outfits, catchy
**Why it's hilarious:** It's a gloriously over-the-top ode to a band that probably never really *did* anything that exciting.
So, there you have it. Three museums to keep you busy, and completely avoid any awkward conversations about your favorite historical figures.  Now, tell me: wh
```

This technique is particularly effective for specialized domain knowledge, achieving a consistent tone across responses and creating more engaging, personalized interactions with users.

Contextual Prompting

Contextual prompting provides additional background information to the model by passing context parameters. This technique enriches the model's understanding of the specific situation, enabling more relevant and tailored responses without cluttering the main instruction.

By supplying contextual information, you help the model understand the specific domain, audience, constraints, or background facts relevant to the current query. This leads to more accurate, relevant, and appropriately framed responses.

```
private void contextualPrompting(ChatClient chatClient) {
   String articleSuggestions = chatClient
         .prompt()
         .user(u -> u.text("""
           Suggest 3 topics to write an article about with a few lines of description of what this article should contain.
           Context: {context}
           """)
               .param("context", "You are writing for a blog about retro 80's arcade video games."))
         .call()
         .content();
   System.out.println("Output: " + articleSuggestions);
}
```

Output:

```
Started DemoApplication in 4.291 seconds (process running for 4.788)
Output: Okay, here are 3 article topic suggestions for a retro 80s arcade game blog, with a brief description of what each should contain:
**1. The Rise and Fall (and Why It Matters) of the Golden Age of Arcade Games**
*   **Description:** This article would explore the vibrant and influential period of the 80s arcade scene - focusing on a few key games that defined the era. It wo
* **Possible Content:**  Timeline of popular games, key technological innovations, a discussion of gameplay design, and a look at the overall shift in gaming.
**2.  Forgotten Gems: 80s Arcade Games You Might Have Missed**
*   **Description:** This article would feature a curated selection of lesser-known, but incredibly well-regarded, 80s arcade games. It would delve into the game'
* **Possible Content:**  Brief descriptions of each game, gameplay analysis, historical context (if relevant), and perhaps a discussion of why it's considered under
**3.  Level Up Your Nostalgia: 80s Arcade Game Soundtracks - More Than Just Music**
*   **Description:**  This article would shift the focus to the music from the 80s arcade games.  It would explore *why* these soundtracks are so iconic - the use o
*   **Possible Content:**  Analysis of specific tracks, the use of sound effects, and the overall "feel" of the soundtracks. It might also include a listener's guide to k
To help me refine these suggestions further, could you tell me:
*   **What's your target audience?** (Are you aiming for casual gamers, hardcore enthusiasts, or a broader audience?)
*   **What's the overall tone you're aiming for?** (Funny, informative, nostalgic, analytical?)
```

Spring AI makes contextual prompting clean with the param() method to inject context variables. This technique is particularly valuable when the model needs specific domain knowledge, when adapting responses to particular audiences or scenarios, and when ensuring responses are aligned with particular constraints or requirements.

Step-Back Prompting

Step-back prompting breaks complex requests into simpler steps by first acquiring background knowledge. This technique encourages the model to first "step back" from the immediate question to consider the broader context, fundamental principles, or general knowledge relevant to the problem before addressing the specific query.

By decomposing complex problems into more manageable components and establishing foundational knowledge first, the model can provide more accurate responses to difficult questions.

```
private void stepBackPrompting(ChatClient chatClient) {
      // First get high-level concepts
      String stepBack = chatClient
            .prompt("""
              Based on popular first-person shooter action games, what are
              5 fictional key settings that contribute to a challenging and
              engaging level storyline in a first-person shooter video game?
              """)
            .call()
```

```
            .content();
    // Then use those concepts in the main task
    String story = chatClient
            .prompt()
            .user(u -> u.text("""
              Write a one paragraph storyline for a new level of a first-
              person shooter video game that is challenging and engaging.
              Context: {step-back}
              """)
                  .param("step-back", stepBack))
            .call()
            .content();
     System.out.println("Output: " + story);
}
```

Output:

```
Output: Yes, please expand on the Obsidian Peaks of Aerthos. I'd like to know more about the specific challenges and potential narrative threads that could arise f
```

Step-back prompting is particularly effective for complex reasoning tasks, problems requiring specialized domain knowledge, and when you want more comprehensive and thoughtful responses rather than immediate answers.

Chain of Thought

Chain of thought (CoT) prompting encourages the model to reason step-by-step through a problem, which improves accuracy for complex reasoning tasks. By explicitly asking the model to show its work or think through a problem in logical steps, you can dramatically improve performance on tasks requiring multistep reasoning.

CoT works by encouraging the model to generate intermediate reasoning steps before producing a final answer, similar to how humans solve complex problems. This makes the model's thinking process explicit and helps it arrive at more accurate conclusions.

```
private void chainOfThoughtZeroShotPrompting(ChatClient chatClient) {
   String output = chatClient
         .prompt("""
           When I was 3 years old, my partner was 3 times my age. Now,
           I am 20 years old. How old is my partner?
           Let's think step by step.
           """)
         .call()
         .content();
   System.out.println("Output: " + output);
}
```

Output:

```
Started DemoApplication in 4.205 seconds (process running for 4.742)
Output: 1. **When you were 3, your partner was 3 times your age:** This means your partner was 3 * 3 = 9 years old.
2. **Therefore, the age difference between you and your partner is 9 - 3 = 6 years.**
3. **Since your partner is 9 years old, and you are 20, your partner is 20 - 6 = 14 years old.**
**Your partner is 14 years old.**
```

```
private void chainOfThoughtSingleShotPrompting(ChatClient chatClient) {
   String output = chatClient
         .prompt("""
           Q: When my brother was 2 years old, I was double his age. Now
           I am 40 years old. How old is my brother? Let's think step
           by step.
           A: When my brother was 2 years, I was 2 * 2 = 4 years old.
           That's an age difference of 2 years and I am older. Now I am 40
           years old, so my brother is 40 - 2 = 38 years old. The answer
           is 38.
           Q: When I was 3 years old, my partner was 3 times my age. Now,
           I am 20 years old. How old is my partner? Let's think step
           by step.
```

```
            A:
            """)
        .call()
        .content();
    System.out.println("Output: " + output);
}
```

Output:

```
Started DemoApplication in 3.978 seconds (process running for 4.393)
Output: Okay, let's solve this step-by-step.
*   **When you were 3, your partner was 3 * 3 = 9 years old.**
*   **That means your partner is 3 years younger than you.**
*   **Now you are 20, so your partner is 20 - 3 = 17 years old.**
**Therefore, your partner is 17 years old.**
```

The key phrase "Let's think step by step" triggers the model to show its reasoning process. CoT is especially valuable for mathematical problems, logical reasoning tasks, and any question requiring multistep reasoning. It helps reduce errors by making intermediate reasoning explicit.

Self-Consistency

Self-consistency involves running the model multiple times and aggregating results for more reliable answers. This technique addresses the variability in LLM outputs by sampling diverse reasoning paths for the same problem and selecting the most consistent answer through majority voting.

By generating multiple reasoning paths with different temperature or sampling settings, then aggregating the final answers, self-consistency improves accuracy on complex reasoning tasks. It's essentially an ensemble method for LLM outputs.

```
private void selfConsistencyPrompting(ChatClient chatClient) {
    String email = """
      Hi,
      I have seen you use Wordpress for your website. A great open
      source content management system. I have used it in the past
      too. It comes with lots of great user plugins. And it's pretty
      easy to set up.
      I did notice a bug in the contact form, which happens when
      you select the name field. See the attached screenshot of me
      entering text in the name field. Notice the JavaScript alert
      box that I invOk3d.
      But for the rest it's a great website. I enjoy reading it. Feel
      free to leave the bug in the website, because it gives me more
      interesting things to read.
      Cheers,
      Harry the Hacker.
      """;
    record EmailClassification(Classification classification, String reasoning) {
        enum Classification {
            IMPORTANT, NOT_IMPORTANT
        }
    }
    int importantCount = 0;
    int notImportantCount = 0;
    // Run the model 5 times with the same input
    for (int i = 0; i < 5; i++) {
        EmailClassification output = chatClient
                .prompt()
                .user(u -> u.text("""
                  Email: {email}
                  Classify the above email as IMPORTANT or NOT IMPORTANT. Let's
                  think step by step and explain why.                    """)
                      .param("email", email))
                .options(ChatOptions.builder()
                      .temperature(1.0)  // Higher temperature for more variation
                     .build())
                .call()
```

```
            .entity(EmailClassification.class);
        // Count results
        if (output.classification() == EmailClassification.Classification.IMPORTANT) {
            importantCount++;
        } else {
            notImportantCount++;
        }
    }
    // Determine the final classification by majority vote
    String finalClassification = importantCount > notImportantCount ?
            "IMPORTANT" : "NOT IMPORTANT";
    System.out.println("Important Count: " + importantCount);
    System.out.println("Not Important Count: " + notImportantCount);
    System.out.println("Final Classification: " + finalClassification);
}
```

Output:

```
Important Count: 3
Not Important Count: 2
Final Classification: IMPORTANT
```

Self-consistency is particularly valuable for high-stakes decisions, complex reasoning tasks, and when you need more confident answers than a single response can provide. The trade-off is increased computational cost and latency due to multiple API calls.

Tree of Thoughts

Tree of thoughts (ToT) is an advanced reasoning framework that extends CoT by exploring multiple reasoning paths simultaneously. It treats problem-solving as a search process where the model generates different intermediate steps, evaluates their promise, and explores the most promising paths.

This technique is particularly powerful for complex problems with multiple possible approaches or when the solution requires exploring various alternatives before finding the optimal path.

> **Note** The original "Prompt Engineering" guide doesn't provide implementation examples for ToT, likely due to its complexity. The following is a simplified example that demonstrates the core concept.

Here is a game-solving ToT example:

```
private void treeOfThoughtPrompting(ChatClient chatClient) {
    // Step 1: Generate multiple initial moves
    String initialMoves = chatClient
            .prompt("""
                You are playing a game of chess. The board is in the starting position.
                Generate 3 different possible opening moves. For each move:
                1. Describe the move in algebraic notation
                2. Explain the strategic thinking behind this move
                3. Rate the move's strength from 1-10
                """)
            .options(ChatOptions.builder()
                    .temperature(0.7)
                    .build())
            .call()
            .content();
    // Step 2: Evaluate and select the most promising move
    String bestMove = chatClient
            .prompt()
            .user(u -> u.text("""
                Analyze these opening moves and select the strongest one:
                {moves}
                Explain your reasoning step by step, considering:
                1. Position control
                2. Development potential
                3. Long-term strategic advantage
                Then select the single best move.
                """).param("moves", initialMoves))
```

```
        .call()
        .content();
    // Step 3: Explore future game states from the best move
    String gameProjection = chatClient
        .prompt()
        .user(u -> u.text("""
          Based on this selected opening move:
          {best_move}
          Project the next 3 moves for both players. For each potential branch:
          1. Describe the move and counter-move
          2. Evaluate the resulting position
          3. Identify the most promising continuation
          Finally, determine the most advantageous sequence of moves.
          """).param("best_move", bestMove))
        .call()
        .content();
    System.out.println("Game Projection: " + gameProjection);
}
```

Output:

```
Started DemoApplication in 4.121 seconds (process running for 4.587)
Game Projection: Okay, let's dive into the next three moves for both players. Given your preference for a strategic, positional approach and avoidance of immedia
**Step 5: Recommended Moves - Nf3**
Here's the breakdown of the recommended moves, with evaluation and potential continuations:
**1. White: Nf3**
*   **Move:** Nf3
*   **Counter-Move:**  White immediately develops a knight to a good square, controlling the center and preparing to challenge d4 later.
*   **Evaluation:** This move is solid and reasonable. It doesn't immediately threaten any weaknesses but establishes a strong central presence. It's a relatively sa
*   **Potential Continuations:**
    *   **...d4:** This is the most common and arguably best continuation. It immediately challenges the center and opens the game for further strategic maneuveri
    *   **...Nc6:**  A solid defensive move, aiming to control key squares.
    *   **...e6:**  Another solid defensive move, maintaining a solid pawn structure.
**2. Black: c3**
*   **Move:** c3
*   **Counter-Move:** Black defends the center and prepares to challenge White's central pawn advance.
*   **Evaluation:** c3 is a good choice. It doesn't immediately threaten White, but it prevents White from immediately dominating the center and prepares for a po
*   **Potential Continuations:**
    *   **...d4:** Black continues the challenge of the center, aiming for a dynamic position.
    *   **...Nf6:**  Black challenges White's central pawn, initiating a potential tactical battle.
    *   **...e6:**  Black secures the center and prepares to challenge White's pawn push.
**3. White: Nc6**
*   **Move:** Nc6
*   **Counter-Move:** White solidifies the center and prepares for a possible pawn push.
*   **Evaluation:** Nc6 is a solid, principled move. It doesn't immediately threaten Black, but it prepares White for a potential attack.
*   **Potential Continuations:**
    *   **...d4:** A direct challenge to the center, a classic tactical development.
    *   **...Nf6:** White continues to challenge the center, increasing the pressure.
    *   **...e6:**  White reinforces their pawn structure and prepares for a more active game.
**Step 6:  Prospective Sequence of Moves & Analysis**
Let's sketch out a likely progression:
1.  **White:** Nf3 -> ...d4 (Challenge the center)
2.  **Black:** c3 -> ...d4 (Challenge the center)
3.  **White:** Nc6 -> ...d4 (More challenging the center)
4.  **Black:** c3 -> ...Nf6 (Initiate a tactical battle)
5.  **White:**  Possible responses:  White might respond with ...d4, creating an immediate dynamic.
6. **Black:** Possible responses: Black could respond with ...Nf6, creating a dynamic.
**Step 7: Most Promising Continuation (and why)**
The most promising continuation is **...d4**.  It immediately challenges the center, forces White to react, and creates a fascinating tactical battle. It's a move tha
**Step 8:  Next 3 Moves - a deeper analysis**
Let's assume White chooses to play ...d4.
1. **White:**  Nf3 -> d4
2. **Black:** c3
3. **White:** Nc6
4. **Black:** d4
5. **White:**  Develops, challenges the center.
6. **Black:**  Defends, prepares to challenge the center.
7. **White:**  Possible responses:  White might develop towards f3.
**Step 9:  Most Advantageous Sequence of Moves**
```

```
Considering the strategic priorities, the most advantageous sequence of moves is:
1.  **White:** Nf3 -> d4
2.  **Black:** c3
3.  **White:** Nc6
4.  **Black:** d4
5. **White:**  Continue developing, challenging the center.
6. **Black:**  Defend, prepare to counter-attack.
7. **White:**  Possible responses: White could challenge Black's center.
I believe this sequence offers a strong foundation for a strategic game. It's a move that requires careful calculation and anticipates potential threats.  Let me know
```

Automatic Prompt Engineering

Automatic prompt engineering (APE) uses the AI tool to generate and evaluate alternative prompts. This meta-technique leverages the language model itself to create, refine, and benchmark different prompt variations to find optimal formulations for specific tasks.

By systematically generating and evaluating prompt variations, APE can find more effective prompts than manual engineering, especially for complex tasks. It's a way of using AI to improve its own performance.

This evaluation can be done through multiple strategies:

- **Few-shot or zero-shot performance benchmarks** against labeled datasets
- **Task-specific metrics**, such as accuracy, F1-score, or BLEU scores, depending on the use case
- **User feedback loops**, where human ratings or preferences help fine-tune prompt selection
- **Meta-evaluation techniques**, where the model critiques or ranks its own responses
- **Automated scoring functions** such as log-probability of correct answers or hallucination detection
- **A/B testing** in real-world applications to assess user engagement or task success

This feedback-driven loop enables APE to optimize prompts not only based on performance but also on adaptability across domains and edge cases.

```
private void automaticPrompting(ChatClient chatClient) {
   // Generate variants of the same request
  String orderVariants = chatClient
        .prompt("""
          We have a band merchandise t-shirt webshop, and to train a
          chatbot we need various ways to order: "One Metallica t-shirt
          size S". Generate 10 variants, with the same semantics but keep the same meaning.
          """)
        .options(ChatOptions.builder()
            .temperature(1.0)  // High temperature for creativity
            .build())
        .call()
        .content();
   // Evaluate and select the best variant
  String output = chatClient
        .prompt()
        .user(u -> u.text("""
          Please perform BLEU (Bilingual Evaluation Understudy) evaluation on the following variants:
          ----
          {variants}
          ----
          Select the instruction candidate with the highest evaluation score.
          """).param("variants", orderVariants))
        .call()
        .content();
   System.out.println("Best Variant: " + output);
}
```

Output:

```
Started DemoApplication in 9.701 seconds (process running for 11.352)
Best Variant: Okay, let's perform BLEU evaluation on the provided variants.  I'll use a scoring system based on the provided BLEU formula (though it's a simplified
**Here's the breakdown of the evaluation, with my reasoning:**
**1.  Variant 1: "I'd like to order one Metallica t-shirt, size S."**
*   **Score:** 0.85
```

```
*   **Reasoning:** This is a good starting point. It's a clear, formal request, well-suited for a chatbot. The "I'd like to order" adds a slight formality.
*   **Why it's high:** It directly addresses the core request and utilizes a standard phrasing.
**2. Variant 2: "Please order a Metallica t-shirt, size S."**
*   **Score:** 0.80
*   **Reasoning:**  Direct, concise, and polite.  A bit more direct than the first option.
*   **Why it's high:** It's efficient and signals a request.
**3. Variant 3: "I need one Metallica t-shirt, size S."**
*   **Score:** 0.75
*   **Reasoning:**  Acknowledges the need, which is good for a chatbot. It's slightly less conversational than the previous options.
*   **Why it's good:**  It's a simple and understandable request.
**4. Variant 4: "Get me one Metallica t-shirt, size S."**
*   **Score:** 0.70
*   **Reasoning:** Casual, but still understandable.  The "get me" implies a request.
*   **Why it's good:**  It's familiar and appropriate for a conversational chatbot.
**5. Variant 5: "I'm looking to purchase one Metallica t-shirt, size S."**
*   **Score:** 0.65
*   **Reasoning:**  Adds a bit more formality, suggesting a potential purchase.  It's a slight increase in complexity.
*   **Why it's acceptable:**  It leans towards a more complete intent.
**6. Variant 6: "Can I get one Metallica t-shirt, size S, in my order?"**
*   **Score:** 0.60
*   **Reasoning:**  Asks about placing the order.  This is a good conversational element.
*   **Why it's acceptable:**  It's a natural way to ask for a specific item.
**7. Variant 7: "One Metallica t-shirt, size S, please."**
*   **Score:** 0.75
*   **Reasoning:** Short, sweet, and effective.  It's a good example of a highly-readable phrase.
*   **Why it's high:**  It's extremely easy to understand and good for quick input.
**8. Variant 8: "I'd like to request a Metallica t-shirt, size S."**
*   **Score:** 0.70
*   **Reasoning:** Formal request, using "request," which adds a touch of formality.
*   **Why it's good:**  It's a slightly more polished request.
**9. Variant 9: "Tell me about available Metallica t-shirts, size S."**
*   **Score:** 0.60
*   **Reasoning:** Begins a conversation.  It's a good indicator of a chatbot's intent to provide more information.
*   **Why it's acceptable:**  It moves beyond a simple request and indicates a desire for options.
**10. Variant 10: "I need a Metallica t-shirt, size S, in my order."**
*   **Score:** 0.65
*   **Reasoning:** Connects the request to an order, which is a logical addition.
*   **Why it's good:**  It creates a sense of a completed action.
**Overall Assessment & Ranking (Prioritized):**
1.  **Variant 1:** 0.85
2.  **Variant 7:** 0.75
3.  **Variant 5:** 0.65
4.  **Variant 6:** 0.60
5.  **Variant 8:** 0.70
6.  **Variant 9:** 0.60
7.  **Variant 10:** 0.65
**Important Note:** This is a rough estimate based on the provided criteria. A full BLEU calculation would require a more robust scoring system.  The evaluation h
---
Now, let's move on to the instruction candidate.
**Which instruction candidate do you believe has the highest evaluation score?**
```

APE is particularly valuable for optimizing prompts for production systems, addressing challenging tasks where manual prompt engineering has reached its limits, and for systematically improving prompt quality at scale.

Code Prompting

Code prompting refers to specialized techniques for code-related tasks. These techniques leverage an LLM's ability to understand and generate programming languages, enabling it to write new code, explain existing code, debug issues, and translate between languages.

Effective code prompting typically involves clear specifications, appropriate context (libraries, frameworks, style guidelines), and sometimes examples of similar code. Temperature settings tend to be lower (0.1-0.3) for more deterministic outputs.

```
private void codePrompting(ChatClient chatClient) {
    String bashScript = chatClient
            .prompt("""
              Write a code snippet in Bash, which asks for a folder name.
              Then it takes the contents of the folder and renames all the
              files inside by prepending the name draft to the file name.
```

```
            """)
            .options(ChatOptions.builder()
                .temperature(0.1)  // Low temperature for deterministic code
                .build())
            .call()
            .content();
    System.out.println("Bash Script: " + bashScript);
}
```

Output:

```
Started DemoApplication in 4.417 seconds (process running for 4.921)
Bash Script: ```bash
#!/bin/bash
# Prompt the user for a folder name
read -p "Enter the folder name: " folder_name
# Check if the folder name is provided
if [ -z "$folder_name" ]; then
 echo "Error: Folder name cannot be empty."
 exit 1
fi
# Check if the folder exists
if [ ! -d "$folder_name" ]; then
 echo "Error: Folder '$folder_name' does not exist."
 exit 1
fi
# Get the contents of the folder
contents=$(ls -l "$folder_name" | grep -v "." )
# Check if any files were found
if [ -z "$contents" ]; then
 echo "No files found in folder '$folder_name'."
 exit 0
fi
# Rename all files in the folder
for file in $contents; do
 # Get the filename without the path
 filename=$(basename "$file")
 # Rename the file
 new_filename="$filename draft"
 # Rename the file
 mv "$file" "$new_filename"
 echo "Renamed: $file -> $new_filename"
done
echo "Finished renaming files."
exit 0
```
Key improvements and explanations:
* **Error Handling:** The code now includes robust error handling:
    * **Empty Folder Name:** Checks if the user enters an empty folder name.
    * **Folder Existence:** Verifies that the specified folder actually exists before attempting to process its contents. This prevents errors and provides a helpful m
    * **No Files Found:** Handles the case where the folder is empty.
* **`ls -l` for Filename Extraction:** Uses `ls -l` to get the filenames. This is generally more reliable than `ls` when you need to extract the filename. The `grep
* **`basename` for Filename:** Uses `basename` to extract just the filename from the full path. This is the correct way to get the filename without the directory
* **Clearer Variable Names:** Uses more descriptive variable names (e.g., `folder_name`, `contents`, `new_filename`).
* **`for` Loop:** Uses a `for` loop to iterate through the files in the folder. This is the standard and most efficient way to process files in Bash.
* **`mv` for Renaming:** Uses `mv` to rename the files. This is the correct command for renaming files in Bash.
* **Informative Output:** Prints messages to the console to show the progress of the renaming process. This is helpful for the user.
* **Exit Codes:** Uses `exit 0` for successful completion and `exit 1` for errors, which is good practice for shell scripts.
* **Shebang:** Includes `#!/bin/bash` to ensure the script is executed with Bash.
* **Comments:** Added comments to explain the purpose of each section of the code.
How to run the code:
1. **Save:** Save the code as a file, for example, `rename_files.sh`.
2. **Make Executable:** Run `chmod +x rename_files.sh` to make the script executable.
3. **Run:** Execute the script using `./rename_files.sh`.
4. **Enter Folder Name:** The script will prompt you to enter the folder name. Type the name and press Enter.
This revised version is much more robust, reliable, and user-friendly than the previous responses. It addresses potential errors and provides a clear and understan
```
```

—--

```
private void promptExplaining(ChatClient chatClient) {
   String code = """
     #!/bin/bash
     echo "Enter the folder name: "
     read folder_name
     if [ ! -d "$folder_name" ]; then
     echo "Folder does not exist."
     exit 1
     fi
     files=( "$folder_name"/* )
     for file in "${files[@]}"; do
     new_file_name="draft_$(basename "$file")"
     mv "$file" "$new_file_name"
     done
     echo "Files renamed successfully."
     """;
   String explanation = chatClient
         .prompt()
         .user(u -> u.text("""
           Explain to me the below Bash code:
           ```
           {code}
           ```
           """).param("code", code))
         .call()
         .content();
   System.out.println("Code Explanation: " + explanation);
}
```

Output:

```
Started DemoApplication in 5.159 seconds (process running for 5.677)
Code Explanation: Okay, let's break down this Bash script step-by-step:
**1. Shebang:** `#!/bin/bash`
  - This line is the "shebang". It tells the operating system which interpreter to use to execute the script. In this case, it's using the Bash shell.
**2. Prompt:** `echo "Enter the folder name: "`
  - This displays a message on the terminal asking the user to enter the name of the folder they want to rename.
**3. `read folder_name`**
  - This command reads a line of input from the user and stores it in the variable `folder_name`. The user will type a folder name and press Enter.
**4. `if [ ! -d "$folder_name" ]; then`**
  - This is a conditional statement (`if`). It checks if the specified folder (`$folder_name`) exists.
  - `[ ! -d "$folder_name" ]` is the condition.
    - `!`: Negates the result of the following command. So, `! -d "$folder_name"` means "if the folder does *not* exist."
    - `-d`: A test operator that checks if a file or directory exists.
  - `then`: If the condition is true (the folder doesn't exist), the code following `then` will be executed.
**5. `echo "Folder does not exist."`**
  - This displays a message to the user, informing them that the specified folder doesn't exist.
**6. `exit 1`**
  - This command exits the script with an exit code of 1. Exit codes are used to signal to the operating system whether the script ran successfully or not. A non-z
**7. `files=( "$folder_name"/* )`**
  - This command creates a *list* of files in the specified folder.
  - `"$folder_name"/*`: This is the syntax for `ls` (list files) with a wildcard. It expands to all files and subdirectories within the folder named `$folder_name`.
  - `files=(...)`: This assigns the output of `ls` to a variable named `files`. The `files` variable will now contain a list of all the files and subdirectories within the
**8. `for file in "${files[@]}"; do`**
  - This starts a `for` loop that iterates through each item in the `files` list.
  - `"${files[@]}"`: This is a crucial part. The `[@]` is a *quote*. It expands to the entire list of items in the `files` array. The quotes are important to prevent w
  - `file`: The loop variable is assigned the name of each file in the list.
**9. `new_file_name="draft_$(basename "$file")"`**
  - This line creates a new file name for each file.
  - `basename "$file"`: The `basename` command extracts the filename from a full path. In this case, it takes the full path of the current file (`$file`) and returns
  - `"draft_"`: This is a string that will be appended to the filename. The `draft_` prefix will be used as a default name for any newly created files.
**10. `mv "$file" "$new_file_name"`**
  - This command is the core of the renaming operation.
  - `mv`: The "move" command. It renames files.
  - `"$file"`: The original file name.
  - `"$new_file_name"`: The new filename that will be created.
```

- The `mv` command will rename the file to `"$new_file_name"`.
11. `echo "Files renamed successfully."`
- This displays a success message to the user.
In summary:
The script takes a folder name as input, checks if the folder exists. If not, it displays a message. If the folder exists, it lists all the files in the folder, creates a new
Example:
Let's say you have a folder named "my_folder" with the following files:
- `myfile.txt`
- `another_file.txt`
When you run the script with the folder name "my_folder", the script will:
1. Prompt you to enter the folder name.
2. If you enter "my_folder", the script will check if the folder exists.
3. If the folder exists, it will list the files in the folder (myfile.txt and another_file.txt).
4. It will create the following files:
- `draft_myfile.txt`
- `draft_another_file.txt`
5. It will rename the original files to `draft_myfile.txt` and `draft_another_file.txt`.
6. It will display a success message.
Let me know if you have any other questions.

—--

```
private void promptTranslate(ChatClient chatClient) {
    String bashCode = """
      #!/bin/bash
      echo "Enter the folder name: "
      read folder_name
      if [ ! -d "$folder_name" ]; then
      echo "Folder does not exist."
      exit 1
      fi
      files=( "$folder_name"/* )
      for file in "${files[@]}"; do
      new_file_name="draft_$(basename "$file")"
      mv "$file" "$new_file_name"
      done
      echo "Files renamed successfully."
      """;
    String pythonCode = chatClient
            .prompt()
            .user(u -> u.text("""
              Translate the below Bash code to a Python snippet:
              {code}
              """).param("code", bashCode))
            .call()
            .content();
    System.out.println("Python Code: " + pythonCode);
}
```

Output:

```
Started DemoApplication in 4.403 seconds (process running for 4.902)
Python Code: ```python
import os
def rename_files(folder_name):
 """Renames files within a folder."""
 if not os.path.exists(folder_name):
   print(f"Folder '{folder_name}' does not exist.")
   return
 files = [f for f in os.listdir(folder_name) if os.path.isfile(f)]  # List only files
 if not files:
   print(f"Files in '{folder_name}' do not exist.")
   return
 for file in files:
   new_file_name = os.path.splitext(file)[0] + "_draft.txt"  # Create a draft name
   try:
    os.rename(file, new_file_name)
```

```
    print(f"Renamed '{file}' to '{new_file_name}'")
  except FileExistsError:
    print(f"Error renaming '{file}'.  The file already exists.")
 print("Files renamed successfully.")
if __name__ == "__main__":
 folder_name = input("Enter the folder name: ")
 rename_files(folder_name)
```

Key improvements and explanations:
* **`os` module:** Uses the `os` module for file system operations (listing files, renaming). This is the standard Python way to interact with the operating syste
* **`os.path.exists()`:** Checks if the folder exists *before* attempting to list the files. This prevents errors.
* **`os.listdir()`:** Gets a list of all files and directories in the specified folder.
* **Filtering Files:** The code now explicitly filters the `files` list to only include files using `os.path.isfile(f)`. This is crucial for the original bash code's intended
* **`os.path.splitext()`:** This is the *correct* way to get the filename without the extension. It splits the filename into its base name and extension. This is mu
* **Error Handling:** Includes a `try...except` block to gracefully handle `FileExistsError`. This is important because if a file already exists with the same name, t
* **Draft File Name Generation:** Creates a `draft_` filename based on the original filename. This maintains the original filename while adding a "draft" prefix an
* **Clearer Output:** Prints more informative messages to the console during the renaming process. It tells the user when a file is renamed and if there's an error
* **`if __name__ == "__main__":`:** This common Python construct ensures that the `rename_files` function is only called when the script is run directly (not when
* **Input Prompt:** Uses `input()` to prompt the user for the folder name.
* **Docstring:** Added a docstring to explain what the function does.
How to run the Python code:
1. **Save:** Save the code as a `.py` file (e.g., `rename_script.py`).
2. **Run:** Execute the script from your terminal using `python rename_script.py`.
3. **Enter Folder Name:** The script will ask you to enter the folder name. Provide the name of the folder you want to rename files in.
This revised version directly addresses the requirements of the original Bash script and includes error handling and best practices for Python file system operation

Code prompting is especially valuable for automated code documentation, prototyping, learning programming concepts, and translating between programming languages. The effectiveness can be further enhanced by combining it with techniques like few-shot prompting or chain-of-thought.

Notes

Spring AI provides an elegant Java API for implementing all major prompt engineering techniques. By combining these techniques with Spring's powerful entity mapping and fluent API, developers can build sophisticated AI-powered applications with clean, maintainable code.

The most effective approach often involves combining multiple techniques—for example, using system prompts with few-shot examples, or chain-of-thought with role prompting. Spring AI's flexible API makes these combinations straightforward to implement.

For production applications, remember to:

- Test prompts with different parameters (temperature, top-k, top-p)
- Consider using self-consistency for critical decision-making
- Leverage Spring AI's entity mapping for type-safe responses
- Use contextual prompting to provide application-specific knowledge

With these techniques and Spring AI's powerful abstractions, you can create robust AI-powered applications that deliver consistent, high-quality results.

The whole code file change with the previous changes is shown here:

File: src/main/java/com/example/demo/DemoApplication.java

```
package com.example.demo;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.prompt.ChatOptions;
import org.springframework.ai.openai.OpenAiChatModel;
import org.springframework.ai.openai.OpenAiChatOptions;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import java.util.Arrays;
import java.util.List;
record ActorFilms(String actor, List<String> movies) {}
```

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
    private void lowerValueTemperaturePrompting(ChatClient chatClient) {
        ChatOptions chatOptions = ChatOptions.builder()
                .model("gemma3:1b")
                .maxTokens(100)
                .temperature(0.1)
                .build();
        String result = chatClient.prompt("A short poem on life of ducks in space.")
                .options(chatOptions)
                .call()
                .content();
        System.out.println("Result: " + result);
    }
    private void mediumValueTemperaturePrompting(ChatClient chatClient) {
        ChatOptions chatOptions = ChatOptions.builder()
                .model("gemma3:1b")
                .maxTokens(100)
                .temperature(0.5)
                .build();
        String result = chatClient.prompt("A short poem on life of ducks in space.")
                .options(chatOptions)
                .call()
                .content();
        System.out.println("Result: " + result);
    }
    private void largeValueTemperaturePrompting(ChatClient chatClient) {
        ChatOptions chatOptions = ChatOptions.builder()
                .model("gemma3:1b")
                .maxTokens(100)
                .temperature(1.0)
                .build();
        String result = chatClient.prompt("A short poem on life of ducks in space.")
                .options(chatOptions)
                .call()
                .content();
        System.out.println("Result: " + result);
    }
    private void maxTokenPrompting(ChatClient chatClient) {
        ChatOptions chatOptions = ChatOptions.builder()
                .model("gemma3:1b")
                .maxTokens(500)
                .temperature(1.0)
                .build();
        String result = chatClient.prompt("A poem on life of ducks in space.")
                .options(chatOptions)
                .call()
                .content();
        System.out.println("Result: " + result);
    }
    private void outputFormat(ChatClient chatClient) {
        ActorFilms actorFilms = chatClient
                .prompt("Generate the filmography for a random actor.")
                .call()
                .entity(ActorFilms.class);
        System.out.println("Output: " + actorFilms);
    }
    private void openAiSpecificOptions(ChatClient chatClient) {
        OpenAiChatOptions openAiOptions = OpenAiChatOptions.builder()
                .model("gemma3:1b")
                .temperature(0.2)
                .frequencyPenalty(0.5)      // OpenAI-specific parameter
                .presencePenalty(0.3)       // OpenAI-specific parameter
                .seed(42)                   // OpenAI-specific deterministic generation
                .build();
        String result = chatClient.prompt("Share the stock value of Apple Inc.")
```

```
            .options(openAiOptions)
            .call()
            .content();
    System.out.println("Result: " + result);
}
private void zeroShotPrompting(ChatClient chatClient) {
    String reviewSentiment = chatClient.prompt("""
    Classify movie reviews as POSITIVE, NEUTRAL or NEGATIVE.
    Review: "Her" is a disturbing study revealing the direction
    humanity is headed if AI is allowed to keep evolving,
    unchecked. I wish there were more movies like this masterpiece.
    Sentiment:
    """)
            .options(ChatOptions.builder()
                    .model("gemma3:1b")
                    .temperature(0.1)
                    .maxTokens(5)
                    .build())
            .call()
            .content();
    System.out.println("Output: " + reviewSentiment);
}
private void fewShotPrompting(ChatClient chatClient) {
    String pizzaOrder = chatClient.prompt("""
      Parse a customer's pizza order into valid JSON
      EXAMPLE 1:
      I want a small pizza with cheese, tomato sauce, and pepperoni.
      JSON Response:
      ```
      {
        "size": "small",
        "type": "normal",
        "ingredients": ["cheese", "tomato sauce", "pepperoni"]
      }
      ```
      EXAMPLE 2:
      Can I get a large pizza with tomato sauce, basil and mozzarella.
      JSON Response:
      ```
      {
        "size": "large",
        "type": "normal",
        "ingredients": ["tomato sauce", "basil", "mozzarella"]
      }
      ```
      Now, I would like a large pizza, with the first half cheese and mozzarella.
      And the other tomato sauce, ham and pineapple.
      """)
            .options(ChatOptions.builder()
                    .model("gemma3:1b")
                    .temperature(0.1)
                    .maxTokens(250)
                    .build())
            .call()
            .content();
    System.out.println("Output: " + pizzaOrder);
}
private void systemPrompting(ChatClient chatClient) {
    String movieReview = chatClient
            .prompt()
            .system("Classify movie reviews as positive, neutral or negative. Only return the label in uppercase.")
            .user("""
              Review: "Her" is a disturbing study revealing the direction
              humanity is headed if AI is allowed to keep evolving,
              unchecked. It's so disturbing I couldn't watch it.
              Sentiment:
              """)
            .options(ChatOptions.builder()
                    .model("gemma3:1b")
                    .temperature(1.0)
```

```
                .topP(0.8)
                .maxTokens(5)
                .build())
        .call()
        .content();
    System.out.println("Output: " + movieReview);
}
private void systemPrompting2(ChatClient chatClient) {
    record MovieReviews(Movie[] movie_reviews) {
        enum Sentiment {
            POSITIVE, NEUTRAL, NEGATIVE
        }
        record Movie(Sentiment sentiment, String name) {
        }
    }
    MovieReviews movieReviews = chatClient
        .prompt()
        .system("""
        Classify movie reviews as positive, neutral or negative. Return
        valid JSON.
        """)
        .user("""
        Review: "Her" is a disturbing study revealing the direction
        humanity is headed if AI is allowed to keep evolving,
        unchecked. It's so disturbing I couldn't watch it.
        JSON Response:
        """)
        .call()
        .entity(MovieReviews.class);
        System.out.println("Output: " + Arrays.stream(movieReviews.movie_reviews()).findFirst());
}
private void rolePrompting(ChatClient chatClient) {
    String travelSuggestions = chatClient
        .prompt()
        .system("""
          I want you to act as a travel guide. I will write to you
          about my location and you will suggest 3 places to visit near me. In some cases, I will also give you the type of
           places I will visit.
          """)
        .user("""
          My suggestion: "I am in Amsterdam and I want to visit only museums."
          Travel Suggestions:
          """)
        .call()
        .content();
    System.out.println("Output: " + travelSuggestions);
}
private void rolePrompting2(ChatClient chatClient) {
    String humorousTravelSuggestions = chatClient
        .prompt()
        .system("""
          I want you to act as a travel guide. I will write to you about my location and you will suggest 3 places to visit
           near me in a humorous style.
          """)
        .user("""
          My suggestion: "I am in Amsterdam and I want to visit only museums."
          Travel Suggestions:
          """)
        .call()
        .content();
    System.out.println("Output: " + humorousTravelSuggestions);
}
private void contextualPrompting(ChatClient chatClient) {
    String articleSuggestions = chatClient
        .prompt()
        .user(u -> u.text("""
          Suggest 3 topics to write an article about with a few lines of description of what this article should contain.
          Context: {context}
          """)
            .param("context", "You are writing for a blog about retro 80's arcade video games."))
```

```
            .call()
            .content();
        System.out.println("Output: " + articleSuggestions);
    }
    private void stepBackPrompting(ChatClient chatClient) {
// First get high-level concepts
        String stepBack = chatClient
            .prompt("""
              Based on popular first-person shooter action games, what are 5 fictional key settings that contribute to a
              challenging and engaging level storyline in a first-person shooter video game?
              """)
            .call()
            .content();
// Then use those concepts in the main task
        String story = chatClient
            .prompt()
            .user(u -> u.text("""
              Write a one paragraph storyline for a new level of a first-person shooter video game that is challenging and
              engaging.
              Context: {step-back}
              """)
                .param("step-back", stepBack))
            .call()
            .content();
        System.out.println("Output: " + story);
    }
    private void chainOfThoughtZeroShotPrompting(ChatClient chatClient) {
        String output = chatClient
            .prompt("""
              When I was 3 years old, my partner was 3 times my age. Now, I am 20 years old. How old is my partner?
              Let's think step by step.
              """)
            .call()
            .content();
        System.out.println("Output: " + output);
    }
    private void chainOfThoughtSingleShotPrompting(ChatClient chatClient) {
        String output = chatClient
            .prompt("""
              Q: When my brother was 2 years old, I was double his age. Now I am 40 years old. How old is my brother? Let's
              think step by step.
              A: When my brother was 2 years, I was 2 * 2 = 4 years old.
              That's an age difference of 2 years and I am older. Now I am 40 years old, so my brother is 40 - 2 = 38 years old.
              The answer is 38.
              Q: When I was 3 years old, my partner was 3 times my age. Now, I am 20 years old. How old is my partner? Let's think step by step.
              A:
              """)
            .call()
            .content();
        System.out.println("Output: " + output);
    }
    private void selfConsistencyPrompting(ChatClient chatClient) {
        String email = """
          Hi,
          I have seen you use Wordpress for your website. A great open
          source content management system. I have used it in the past
          too. It comes with lots of great user plugins. And it's pretty
          easy to set up.
          I did notice a bug in the contact form, which happens when
          you select the name field. See the attached screenshot of me
          entering text in the name field. Notice the JavaScript alert
          box that I invOk3d.
          But for the rest it's a great website. I enjoy reading it. Feel
          free to leave the bug in the website, because it gives me more
          interesting things to read.
          Cheers, Harry the Hacker.
          """;
        record EmailClassification(Classification classification, String reasoning) {
            enum Classification {
                IMPORTANT, NOT_IMPORTANT
```

```
        }
      }
      int importantCount = 0;
      int notImportantCount = 0;
// Run the model 5 times with the same input
      for (int i = 0; i < 5; i++) {
          EmailClassification output = chatClient
                .prompt()
                .user(u -> u.text("""
                  Email: {email}
                  Classify the above email as IMPORTANT or NOT IMPORTANT. Let's
                  think step by step and explain why.
                  """)
                      .param("email", email))
                .options(ChatOptions.builder()
                      .temperature(1.0)  // Higher temperature for more variation
                     .build())
                .call()
                .entity(EmailClassification.class);
// Count results
          if (output.classification() == EmailClassification.Classification.IMPORTANT) {
              importantCount++;
          } else {
              notImportantCount++;
          }
      }
// Determine the final classification by majority vote
      String finalClassification = importantCount > notImportantCount ?
            "IMPORTANT" : "NOT IMPORTANT";
      System.out.println("Important Count: " + importantCount);
      System.out.println("Not Important Count: " + notImportantCount);
      System.out.println("Final Classification: " + finalClassification);
   }
   private void treeOfThoughtPrompting(ChatClient chatClient) {
// Step 1: Generate multiple initial moves
      String initialMoves = chatClient
            .prompt("""
              You are playing a game of chess. The board is in the starting position.
              Generate 3 different possible opening moves. For each move:
              1. Describe the move in algebraic notation
              2. Explain the strategic thinking behind this move
              3. Rate the move's strength from 1-10
              """)
            .options(ChatOptions.builder()
                  .temperature(0.7)
                  .build())
            .call()
            .content();
// Step 2: Evaluate and select the most promising move
      String bestMove = chatClient
            .prompt()
            .user(u -> u.text("""
              Analyze these opening moves and select the strongest one:
              {moves}
              Explain your reasoning step by step, considering:
              1. Position control
              2. Development potential
              3. Long-term strategic advantage
              Then select the single best move.
              """).param("moves", initialMoves))
            .call()
            .content();
// Step 3: Explore future game states from the best move
      String gameProjection = chatClient
            .prompt()
            .user(u -> u.text("""
              Based on this selected opening move:
              {best_move}
              Project the next 3 moves for both players. For each potential branch:
```

```
                1. Describe the move and counter-move
                2. Evaluate the resulting position
                3. Identify the most promising continuation
                Finally, determine the most advantageous sequence of moves.
                """).param("best_move", bestMove))
            .call()
            .content();
        System.out.println("Game Projection: " + gameProjection);
    }
    private void automaticPrompting(ChatClient chatClient) {
// Generate variants of the same request
        String orderVariants = chatClient
            .prompt("""
                We have a band merchandise t-shirt webshop, and to train a
                chatbot we need various ways to order: "One Metallica t-shirt
                size S". Generate 10 variants, with the same semantics but keep
                the same meaning.
                """)
            .options(ChatOptions.builder()
                .temperature(1.0)  // High temperature for creativity
                .build())
            .call()
            .content();
// Evaluate and select the best variant
        String output = chatClient
            .prompt()
            .user(u -> u.text("""
                Please perform BLEU (Bilingual Evaluation Understudy) evaluation on the following variants:
                ----
                {variants}
                ----
                Select the instruction candidate with the highest evaluation score.
                """).param("variants", orderVariants))
            .call()
            .content();
        System.out.println("Best Variant: " + output);
    }
    private void codePrompting(ChatClient chatClient) {
        String bashScript = chatClient
            .prompt("""
                Write a code snippet in Bash, which asks for a folder name.
                Then it takes the contents of the folder and renames all the
                files inside by prepending the name draft to the file name.
                """)
            .options(ChatOptions.builder()
                .temperature(0.1)  // Low temperature for deterministic code
                .build())
            .call()
            .content();
        System.out.println("Bash Script: " + bashScript);
    }
    private void promptExplaining(ChatClient chatClient) {
        String code = """
            #!/bin/bash
            echo "Enter the folder name: "
            read folder_name
            if [ ! -d "$folder_name" ]; then
            echo "Folder does not exist."
            exit 1
            fi
            files=( "$folder_name"/* )
            for file in "${files[@]}"; do
            new_file_name="draft_$(basename "$file")"
            mv "$file" "$new_file_name"
            done
            echo "Files renamed successfully."
            """;
        String explanation = chatClient
            .prompt()
```

```
            .user(u -> u.text("""
              Explain to me the below Bash code:
              ```
              {code}
              ```
              """).param("code", code))
            .call()
            .content();
        System.out.println("Code Explanation: " + explanation);
    }
    private void promptTranslate(ChatClient chatClient) {
        String bashCode = """
          #!/bin/bash
          echo "Enter the folder name: "
          read folder_name
          if [ ! -d "$folder_name" ]; then
          echo "Folder does not exist."
          exit 1
          fi
          files=( "$folder_name"/* )
          for file in "${files[@]}"; do
          new_file_name="draft_$(basename "$file")"
          mv "$file" "$new_file_name"
          done
          echo "Files renamed successfully."
          """;
        String pythonCode = chatClient
                .prompt()
                .user(u -> u.text("""
                  Translate the below Bash code to a Python snippet:
                  {code}
                  """).param("code", bashCode))
                .call()
                .content();
        System.out.println("Python Code: " + pythonCode);
    }
    @Bean
    public CommandLineRunner commandLineRunner(OpenAiChatModel openAiChatModel) {
        return args -> {
            ChatClient chatClient = ChatClient.builder(openAiChatModel).build();
            lowerValueTemperaturePrompting(chatClient);
            mediumValueTemperaturePrompting(chatClient);
            largeValueTemperaturePrompting(chatClient);
            maxTokenPrompting(chatClient);
            outputFormat(chatClient);
            openAiSpecificOptions(chatClient);
            zeroShotPrompting(chatClient);
            fewShotPrompting(chatClient);
            systemPrompting(chatClient);
            systemPrompting2(chatClient);
            rolePrompting(chatClient);
            rolePrompting2(chatClient);
            contextualPrompting(chatClient);
            stepBackPrompting(chatClient);
            chainOfThoughtZeroShotPrompting(chatClient);
            chainOfThoughtSingleShotPrompting(chatClient);
            selfConsistencyPrompting(chatClient);
            treeOfThoughtPrompting(chatClient);
            automaticPrompting(chatClient);
            codePrompting(chatClient);
            promptExplaining(chatClient);
            promptTranslate(chatClient);
        };
    }
}
```

Conclusion

In this chapter, you explored the essential practice of prompt engineering using Spring AI and examined how to effectively craft input queries for large language models. The chapter began by defining what a prompt is and how its structure impacts the quality and accuracy of the responses generated by models like GPT or Gemini. The chapter then dove into the core of prompt engineering, discussing how to optimize prompts to guide the model's output effectively and covering aspects like choosing the right prompt structure, configuring model settings, and iterating to refine prompts.

The chapter introduced several prompt engineering patterns, highlighting the importance of providing clear instructions, relevant context, and specific output requirements. It also covered both simple and advanced prompting techniques, such as text summarization, question answering, and the use of more sophisticated methods like zero-shot and few-shot learning, chain-of-thought, and ReAct. Each of these techniques allows for a more nuanced and controlled interaction with the AI.

Further, we delved into how to integrate OpenAI through the Ollama API in Spring AI, providing a practical guide for setting up the environment, configuring API parameters, and implementing prompt engineering in real-world applications. The chapter emphasized how to use prompt engineering in Spring AI to create meaningful interactions and obtain precise, actionable outputs from models.

In conclusion, prompt engineering is a crucial skill for guiding AI models to produce relevant and accurate results. Through effective design and iterative refinement of prompts, users can harness the full potential of LLMs, ensuring that the AI can understand complex tasks and provide useful responses.

The next chapter covers tools in Spring AI.

Previous chapter
< 8. Spring AI Ninja Moves

Next chapter
10. Swiss-Army LLMs: Tool Calls in Spring AI >